Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with eachother. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Task Configuration

I called it task configuration where we should find a good launch configuration. Here, we calculated the BLOCK_SIZE by dividing the number of possible threads by the number of block per SM.

$$\text{BLOCK\_SIZE} = \frac{\text{max threads per SM}}{\text{max blocks per SM}} = \frac{1024}{16} = 64$$

The GRID_SIZE is calculated by multiplying the number of possible SMs with the number of blocks per SM giving the total number of possible blocks across all SMs.

$$\text{GRID\_SIZE} = \text{number of SMs} \cdot \text{max blocks per SM} = 34 \cdot 24 = 640$$

This results in the most optimal utilization as we use all threads across a block and only launch the most number of blocks without overhead.
It was also given, that the float array should be exactly 40 MB. To get the length of the array, we calculate the following:

$$\text{length} = \frac{40 \text{ MB} \cdot 1024 \cdot 1024}{4 \text{ Bytes}} = 10.485.760$$

To get the number of elements, we first convert 40 MB to Bytes by multiplying it by 1024 twice. The `sizeof(float)` is 4 Bytes, so we devide it by 4 Bytes. We end up with ~10,5 million float numbers in our input array.

Task 1: CPU reduction algorithm

Here, a baseline on CPU is implemented used to compute the results. The reduction algorithm simply returns the sum of the array. The CPU is the slowest test because of sequential operations.

Task 2: GPU reduction with AtomicAdd and global memory

In this task, we try to optimize our CPU implementation by using the parallelism of the GPU. as we can observe, the GPU runtime takes 20 ms less than the CPU implementation (40 compared to 20 ms). This is faster, however, we would expect more speedup due to parallelism. The time however is limited by the read and write operation on the global float value. Each GPU thread tries to read

the current float value, adds up the corresponding element to the global value and stores it again. As all operations are done on the same memory address, we have to use `atomicAdd` as we would have a race condition otherwise. The usage ensures, that no other thread tries to read and/or write the data at the same time. However, this leads to sequential operations which is the reason why to GPU runtime is pretty much the same as the CPU because implementations have sequential operations.

Task 3: GPU reduction using cascaded algorithm

The idea is to reduce the number of `atomicAdd` operations as they do slow down the operation a lot. This has been observed in task 2.
To speed up, we first reduce on thread level, then use a shared float to reduce on block level. In the end, only one thread per block uses the `atomicAdd` function to add the blocks total sum to the global memory. Using this method, the number of operations, which are slowed down due to race conditions is reduced a minimum.
We can see, that this results in an enormous speedup compared to the GPU and CPU implementation (~x100 speedup).